

Database Design Document (DBD)

Overview

This document outlines the professional, production-ready MongoDB schema design for the **Internship Progress Dashboard** project. The system is designed to streamline the internship lifecycle, facilitating task management, report submission, and mentor feedback.

Purpose of Database

The primary purpose of this database is to reliably store, manage, and efficiently retrieve all data related to intern tasks, progress reports, mentor assignments, and performance feedback.

Why MongoDB is Used

MongoDB (a NoSQL document database) is chosen for its flexibility, scalability, and performance in handling the evolving, semi-structured data typical of a dashboard application. Its document-oriented nature allows us to embed relevant data (e.g., embedding user profiles in documents where necessary) to reduce joins and minimize I/O operations, which is crucial for high-read dashboard views.

Data Structure Rationale (Based on Given JSON)

The provided datasets (Users, Tasks, Reports, Feedback) show a clear relational structure. We will adopt a **denormalized design with embedded references** to balance data integrity (references for primary relationships) with read performance (embedding data that changes infrequently or is highly accessed together).

Collections Design (Based on Given Data)

Users Collection

This collection stores all intern and mentor profile information, centralizing authentication and role management.

Real Data Reference Analysis:

The original data includes `name`, `email`, `role`, and `hashed_password`.

Field Name	Data Type	Role Rationale
<code>_id</code>	<code>ObjectId</code>	Primary key
<code>name</code>	<code>String</code>	User's full name
<code>email</code>	<code>String</code>	Unique identifier, used for login
<code>passwordHash</code>	<code>String</code>	Improved: Renamed from <code>hashed_password</code> for clarity. Stores the salted and hashed password (e.g., bcrypt)
<code>role</code>	<code>String (Enum)</code>	Defines access level: 'Intern' or 'Mentor'
<code>studentId</code>	<code>String</code>	Added: Required for Interns.
<code>mentorId</code>	<code>ObjectId</code>	Added: References the <code>_id</code> of the assigned mentor (only for Interns).
<code>skills</code>	<code>[String]</code>	Added: List of technical skills.
<code>createdAt</code>	<code>Date</code>	Timestamp of creation
<code>updatedAt</code>	<code>Date</code>	Timestamp of last modification

Intern vs Mentor Roles:

The `role` field dictates system permissions. Mentors can assign tasks and give feedback, while Interns can view assigned tasks and submit reports.

Authentication Fields:

`email` and `passwordHash` are used for secure authentication.

Profile-Related Fields:

`name`, `studentId`, `skills`, and `mentorId` provide context for the user profile and reporting structure.



 Tasks Collection

Stores details about assigned work units. The design focuses on tracking the task lifecycle.

Real Data Reference Analysis:

The original data included `task_name`, `description`, `status`, `assignee_id`, and `creator_id`.

Field Name	Data Type	Rationale
<code>_id</code>	<code>ObjectId</code>	Primary key
<code>title</code>	<code>String</code>	Task name (renamed from <code>task_name</code>)
<code>description</code>	<code>String</code>	Detailed instructions for the task
<code>status</code>	<code>String (Enum)</code>	Improved: Tracks lifecycle: 'to-do', 'in-progress', 'done', 'archived'
<code>assignedTo</code>	<code>ObjectId</code>	Improved: References the Intern's <code>_id</code> (replaces <code>assignee_id</code>)
<code>assignedBy</code>	<code>ObjectId</code>	Improved: References the Mentor's <code>_id</code> (replaces <code>creator_id</code>)
<code>dueDate</code>	<code>Date</code>	Deadline for task completion
<code>createdAt</code>	<code>Date</code>	Added: Timestamp of task creation
<code>updatedAt</code>	<code>Date</code>	Added: Timestamp of last status change/update

Task Lifecycle:

The `status` field manages the task flow: *to-do* (assigned), *in-progress* (started by intern), *done* (completed by intern).

Relationship with Users:

Tasks are directly linked to both the Intern (`assignedTo`) and the Mentor (`assignedBy`) using `ObjectId` references.

Reports Collection

Stores the weekly or project-based reports submitted by interns.

Real Data Reference Analysis:

The original data showed links to the intern and report content.

Field Name	Data Type	Rationale
<code>_id</code>	<code>ObjectId</code>	Primary key
<code>studentId</code>	<code>ObjectId</code>	References the submitting Intern's <code>_id</code> (replaces direct intern reference)
<code>title</code>	<code>String</code>	Title of the report (e.g., "Week 3 Progress Report")
<code>content</code>	<code>String</code>	The body/details of the submitted report
<code>submissionDate</code>	<code>Date</code>	When the report was submitted
<code>status</code>	<code>String (Enum)</code>	Added: 'submitted', 'under-review', 'reviewed'
<code>feedbackRef</code>	<code>ObjectId</code>	Improved: References the <code>_id</code> from the Feedback Collection (for efficient lookup)
<code>createdAt</code>	<code>Date</code>	Timestamp of creation

Weekly Report Submission:

The collection records the report details and submission time.

Relation with Intern:

The `studentId` links the report directly to the Intern who submitted it.

Feedback Collection

Stores mentor feedback for specific reports.

Real Data Reference Analysis:

The original data links feedback to the report and the mentor.

Field Name	Data Type	Rationale
<code>_id</code>	<code>ObjectId</code>	Primary key
<code>reportId</code>	<code>ObjectId</code>	References the Report's <code>_id</code>
<code>mentorId</code>	<code>ObjectId</code>	References the Mentor's <code>_id</code> who provided the feedback
<code>comment</code>	<code>String</code>	The full text of the feedback
<code>rating</code>	<code>Number (1-5)</code>	Added: A quantitative measure of the report quality (1: Poor, 5: Excellent)
<code>createdAt</code>	<code>Date</code>	Added: Timestamp of feedback creation
<code>updatedAt</code>	<code>Date</code>	Added: Timestamp of last modification

Link with Report + Mentor:
`reportId` ensures a one-to-one link with a Report, and `mentorId` records who provided the feedback.

 Relationships (IMPORTANT)

This design uses **reference-based relationships** (using `ObjectId`) to maintain data consistency and scalability, following the "join on read" strategy typical of MongoDB.

Relationship	Type	Collections Involved	Field Name for Reference	Strategy
Mentor-Intern	One-to-Many	Users	<code>mentorId</code> (in Intern document)	Storing the Mentor's <code>_id</code> in the Intern's document.
Intern-Task	One-to-Many	Users, Tasks	<code>assignedTo</code> (in Task document)	Storing the Intern's <code>_id</code> in the Task document.
Intern-Report	One-to-Many	Users, Reports	<code>studentId</code> (in Report document)	Storing the Intern's <code>_id</code> in the Report document.

Relationship	Type	Collections Involved	Field Name for Reference	Strategy
Report-Feedback	One-to-One	Reports, Feedback	<code>feedbackRef</code> (in Report document) and <code>reportId</code> (in Feedback document)	Two-way referencing for quick lookups from both sides.

ObjectId Referencing Strategy:

By storing the `_id` of the related document, we ensure that data remains normalized where necessary (e.g., when a User's email changes, we only update the Users collection, and the references in Tasks and Reports remain valid).



Improved Schema Design (Mongoose)

These schemas use Mongoose conventions, including `type`, `required`, `default`, and references (`ref`), to enforce structure and data validation.

User Schema

```
{
  "name": { "type": "String", "required": true },
  "email": { "type": "String", "required": true, "unique": true },
  "passwordHash": { "type": "String", "required": true },
  "role": {
    "type": "String",
    "enum": ["Intern", "Mentor"],
    "default": "Intern",
    "required": true
  },
  "studentId": { "type": "String", "unique": true, "sparse": true },
  "mentorId": { "type": "ObjectId", "ref": "User", "required": false },
}
```

```
"skills": { "type": "[String]", "default": [] },  
  
"profileImage": { "type": "String" },  
  
"status": { "type": "String", "enum": ["Active", "Inactive"], "default": "Active" },  
  
"timestamps": true  
  
}
```

Task Schema

```
{  
  
  "title": { "type": "String", "required": true },  
  
  "description": { "type": "String", "required": true },  
  
  "status": {  
  
    "type": "String",  
  
    "enum": ["to-do", "in-progress", "done", "archived"],  
  
    "default": "to-do"  
  
  },  
  
  "assignedTo": { "type": "ObjectId", "ref": "User", "required": true },  
  
  "assignedBy": { "type": "ObjectId", "ref": "User", "required": true },  
  
  "dueDate": { "type": "Date" },  
  
  "timestamps": true  
  
}
```

Report Schema

```
{  
  
  "studentId": { "type": "ObjectId", "ref": "User", "required": true },  
  
  "title": { "type": "String", "required": true },  
  
}
```

```
"content": { "type": "String", "required": true },

"submissionDate": { "type": "Date", "default": "Date.now" },

"status": {

  "type": "String",

  "enum": ["submitted", "under-review", "reviewed"],

  "default": "submitted"

},

"feedbackRef": { "type": "ObjectId", "ref": "Feedback", "required": false },

"timestamps": true

}
```

Feedback Schema

```
{

  "reportId": { "type": "ObjectId", "ref": "Report", "required": true, "unique": true },

  "mentorId": { "type": "ObjectId", "ref": "User", "required": true },

  "comment": { "type": "String", "required": true },

  "rating": { "type": "Number", "min": 1, "max": 5 },

  "timestamps": true

}
```

Indexing Strategy

To optimize query performance for common operations (login, dashboard views, filtering), the following indexes are mandatory.

Collection	Field(s)	Type	Rationale
Users	<code>email</code>	Unique	Login/Authentication and guaranteeing uniqueness.
Users	<code>mentorId</code>	Standard	Highly used for quickly fetching all interns of a specific mentor.
Users	<code>studentId</code>	Unique, Sparse	Fast lookup of interns and ensuring ID uniqueness (Sparse is used as Mentors won't have this field).
Tasks	<code>assignedTo</code>	Standard	Crucial for Interns' dashboards (My Tasks).
Reports	<code>studentId</code>	Standard	Crucial for Interns and Mentors tracking report history.
Reports	<code>status</code>	Standard	For filtering reports needing review (status: 'submitted').
Feedback	<code>reportId</code>	Unique	Ensuring a Report has only one Feedback record.
Tasks	<code>status</code>	Standard	For filtering tasks by their lifecycle stage.

Data Flow (Based on Real System)

This section illustrates the core interactions within the system using the defined collections:

- Mentor Assigns Task:**
 - Action:** Mentor (User) submits a new task form.
 - Result:** A new document is created in the **Tasks Collection**.
 - References Set:** `assignedBy` (Mentor's `_id`) and `assignedTo` (Intern's `_id`) are set. `status` defaults to 'to-do'.
- Intern Updates Task Status:**
 - Action:** Intern (User) clicks "Start Task" or "Mark as Done."
 - Result:** The corresponding document in the **Tasks Collection** is updated.

- **Fields Updated:** `status` is changed (e.g., 'to-do' → 'in-progress' → 'done'), and `updatedAt` is updated.
 - 3. **Intern Submits Report:**
 - **Action:** Intern (User) completes and submits a weekly report.
 - **Result:** A new document is created in the **Reports Collection**.
 - **References Set:** `studentId` (Intern's `_id`) is set. `status` defaults to 'submitted'.
 - 4. **Mentor Gives Feedback:**
 - **Action:** Mentor reviews the report and submits feedback.
 - **Result:** A new document is created in the **Feedback Collection**.
 - **References Set:** `reportId` (Report's `_id`) and `mentorId` (Mentor's `_id`) are set.
 - **Update Report:** The corresponding document in the **Reports Collection** is updated: `status` changes to 'reviewed', and `feedbackRef` is set to the new Feedback `_id`.
-

Security Improvements

Based on the initial data analysis, several security improvements are critical for a production environment:

1. **Fix Plain Password Issue (Hashing Required):**
 - The raw `hashed_password` must be replaced with a robust, salted hashing algorithm like **Bcrypt**. The database will only store `passwordHash`. Password checks must be performed via the hashing library.
 2. **Role-Based Access Control (RBAC):**
 - Implement strict checks using the `role` field.
 - *Interns* can only view tasks/reports/feedback assigned to/submitted by them.
 - *Mentors* can view all data for their assigned interns (using the `mentorId` index) and create/update tasks/feedback.
 3. **Input Validation:**
 - Use the Mongoose schema (e.g., `required: true`, `enum`, `min/max`) to ensure data integrity and prevent injection issues at the application layer.
-

Scalability Improvements

1. **Pagination for Tasks/Reports:**

- As the number of tasks and reports grows, loading all documents will be slow. Implement cursor-based or offset-based pagination on list endpoints (e.g., `/api/tasks?page=2&limit=20`).
2. **Index Optimization:**
- Ensure all common filter and sort queries use a composite index where necessary (e.g., an index on `assignedTo` and `status` in the Tasks collection).
3. **Avoid Duplication:**
- The current design minimizes data duplication by using references (`_id`). Avoid embedding large, frequently changing arrays/objects.
-

⚠ Data Issues & Fixes (CRITICAL SECTION)

Analysis of the provided JSON datasets revealed critical issues addressed in this DBD:

Data Issue (Based on JSON)	Fix/Improvement Applied	Reason
Security Issue: Plain/weak password representation (<code>hashed_password</code>)	Renamed to <code>passwordHash</code> and mandates use of Bcrypt for storage.	Security standard for production systems.
Naming Inconsistency: <code>assignee_id</code> , <code>creator_id</code>	Renamed to <code>assignedTo</code> , <code>assignedBy</code>	Better semantic clarity and adherence to standard naming conventions.
Missing Fields: Lack of Timestamps	Added <code>createdAt</code> and <code>updatedAt</code> to all schemas.	Essential for audit trails, caching, and debugging.
Missing Data: No status tracking for Reports	Added <code>status</code> field ('submitted', 'under-review', 'reviewed') to Reports collection.	Necessary for a dashboard view to track progress flow.
Schema Improvement: Intern-Mentor link not explicit	Added <code>mentorId</code> field in the Users collection for Interns.	Defines the organizational structure required for the dashboard.

🌟 Future Enhancements

The following

collections and features are recommended for future releases:

1. **Notifications Collection:**
 - Stores real-time alerts for users (e.g., "New task assigned," "Feedback received"). Indexed by `userId` and `isRead` status for quick retrieval.
2. **Chat System Collection:**
 - Stores one-on-one mentor-intern message threads. Uses two-way referencing or embedding (if chat volume is low) for efficiency.
3. **Activity Logs Collection:**
 - Records system-wide actions (e.g., "Mentor X updated Task Y"). Crucial for auditing and accountability.
4. **Audit Tracking (Embedded):**
 - For critical documents (Tasks, Reports), add a simple embedded array like `history: [{ user: ObjectId, action: String, timestamp: Date }]` to track changes directly within the document.